

Backend Assignment – Mid-Level (Payment Processing System)

Objective

Design and implement a payment processing system that simulates real-world payment gateway behavior. The solution should demonstrate strong backend fundamentals, system design thinking, and the ability to handle real-world edge cases in distributed systems.

Functional Requirements

Payment Lifecycle

The system should support payment initiation, processing, and status tracking with states such as Pending, Processing, Success, and Failed.

Failure Handling & Retry Logic

Implement retry logic with limited configurable attempts and an appropriate strategy such as exponential backoff. Ensure no duplicate or inconsistent states.

Idempotency

Ensure repeated requests do not create duplicate payments or multiple processing. Design a mechanism to safely handle repeated operations.

Concurrency Control

Prevent parallel processing of the same payment and handle race conditions effectively.

External Gateway Simulation

Simulate an external provider with random success, failure, delays, and timeouts. Handle all scenarios gracefully.

Webhook / Callback Handling

Handle asynchronous updates including early callbacks, duplicate callbacks, and conflicting states.

Data Consistency

Ensure consistent state management and proper recovery from partial failures.

Technical Expectations

Code Quality

Clean, modular, and maintainable code with proper separation of concerns.

Error Handling

Handle edge cases such as timeouts, failures, invalid inputs, and race conditions.

Logging & Observability

Log payment lifecycle events, retries, and failures for traceability.

Testing

Cover core flow, retry logic, failure scenarios, and edge cases with tests.

Additional Expectations

Demonstrate awareness of scalability, reliability, and real-world system design considerations.

Bonus (Optional)

Queue-based retry handling, resilience patterns (e.g., circuit breaker), rate limiting, and API documentation.

Evaluation Criteria

Criteria	Focus
Problem Understanding	Completeness of payment flow
System Design Thinking	Handling real-world scenarios
Retry & Failure Handling	Robustness
Concurrency & Idempotency	Correctness under load
Code Quality	Readability and maintainability
Testing	Coverage and depth
Edge Case Handling	Practical reliability